

Joseph Cabrera & Danny Ouyang

ESE224.01

10 May 2026

Final Project

Introduction

StockSim is a historical market analyzer and trading strategy simulator developed in C++. The purpose of this project is to apply the concepts we learned in this class regarding data structures and object-oriented programming to a real-life application. The program loads historical stock data from Yahoo Finance and allows the user to analyze stock performance, compare trading strategies, and investment algorithms over extended periods of time. The implemented system supports stocks and ETFs while also providing tools for backtesting, portfolio tracking, and history management.

This project significantly improves financial analysis by making data organization and traversal more efficient, as datasets contain thousands of financial records. By implementing linked lists, binary search trees, stacks, and queues, this project demonstrates how combining different data structures is critical in solving computational problems in the real world. Algorithmic trading strategies (Fixed SIP, Golden Cross, Momentum, Dynamic SIP) are also explored and implemented.

The dataset used in the project consists of historical Yahoo Finance CSV files for SPX, AMZN, and NVDA. Each CSV file contains daily trading information, including the date, various price values, and trading volume. This CSVParser dynamically maps column headers to support flexible CSV formatting, as header labeling varies across different stocks. The data is

also filtered to the 2000-2020 range. This parsed data is stored in a linked list structure with respect to PriceHistory for analysis and simulation in main.cpp.

AI Usage Log - Introduction

No AI used.

Data Structure Design

Linked List - PriceHistory

The PriceHistory class uses a doubly linked list to store historical stock prices parsed by the CSVParser class. Each node stores one trading day's data, the date, open, high, low, close, and volume values. The doubly linked list structure was chosen because stock data is naturally sequential, requiring frequent traversals chronologically during simulations. The class maintains both head and tail pointers to allow efficient forward and reverse movement of "current." In a singly linked list, which just has "head", reverse iterator traversal would be inefficient because it would require repeatedly scanning forward through the list. Custom and reverse iterator classes are also implemented to simplify traversal operations.

Appending a new trading day occurs in constant time because the tail pointer is maintained throughout the list. Searching by date and printing ranges requires linear traversal due to the fact that linked lists do not provide direct indexing. Despite these limitations, the structure is well-suited for the historical financial data because insertions and traversal are frequently utilized by trading strategies. Overall, appending operates in $O(1)$ time while searching and range printing operate in $O(n)$ time.

Binary Search Tree - StockBST

The StockBST class stores stock performance and trading strategy results ordered by numerical values, such as the annual return of final portfolio outputs. Recursive insertion functions maintain BST ordering properties, where smaller values are inserted into the left subtree and larger values into the right subtree. In addition, duplicate values are inserted into the right subtree to maintain consistency (tie-breaking convention).

The BST supports searching, ranking, and range queries in the system. Inorder traversal automatically prints results in sorted ascending order based on performance. This is very useful when comparing trading strategies or annual returns. The BST is also used in the bonus Dynamic SIP parameter sweep to rank threshold performances and determine the best-performing strategy. The inorder traversal of the BST ranks the sweeps correctly without needing a separate sort function.

Insertion and searching operate in $O(\log n)$ average time and may even degrade to linear time $O(n)$ if the tree becomes unbalanced. Range searches may also require linear time based on worst-case scenarios. The BST is appropriate because the application requires frequent ranking and comparison among performance values, making sorted traversal crucial.

Circular Queue - GoldenCrossStrategy

The CircularQueue class is used in the Golden Cross strategy to calculate rolling moving averages. The circular queue stores only the most recent price values while overwriting the oldest entries when full. The modular arithmetic function is used to wrap indices around the fixed-size buffer.

The structure is ideal for moving-average calculations because financial indicators require only a fixed-size rolling capacity rather than unlimited historical storage. The queue

allows constant-time insertion and removal without shifting elements. Enqueue, dequeue, and peek operations all operate in constant time, while average calculations require linear time traversal of the contents in the queue. Thus, the circular queue was implemented because it minimizes memory while efficiently supporting rolling financial calculations.

Stack - TradeStack

The TradeStack class stores trade history using a linked list based stack implementation. The stack supports undo functionality by reversing the most recent transaction using its LIFO structure. Each node stores a TradeRecord object containing trade details such as ticker, action, shares, price, and date.

Push and pop operations both execute in $O(1)$ constant time because all the edits occur at the top of the stack. The LIFO structure is appropriate because undo operations typically reverse the most recent trade before older trades. The usage of a stack allows the system to maintain a complete trade history while supporting fast reversal of accidental or incorrect trades.

Queue - OrderQueue

The OrderQueue class stores pending market and limit orders using a FIFO structure. Orders are executed in the same order they are inputted, which accurately models real-world order systems. The queue uses dynamically allocated linked nodes with front and back pointers for efficient insertion and removal.

Enqueue and dequeue operations both operate in constant time because only front and back pointers require editing. The queue structure is appropriate because pending orders should execute sequentially in chronological order. Overall, this design allows the system to simulate delayed orders and maintain realistic trading behavior.

AI Usage Log - Data Structure Design

Prompts: How do I append nodes to a linked list; Can you give me the pseudocode for insertHelper; How do I avoid memory leaks?

AI Output: Walk to tail, then update tail->next and reassign tail for O(1) PriceHistory append, pseudocode used recursive BST insert descending left or right based on key values, creates node at empty; To avoid memory leaks, every class needs a destructor to free dynamically allocated memory like nodes and buffers.

Modification: Used pointers to implement linked list and BST since we cannot use STL; Added destructors to each class (PriceHistory, CircularQueue, TradeStack, OrderQueue, StockBST) to free all dynamically allocated memory.

Trading Strategies***Fixed SIP Strategy***

The Fixed SIP strategy is the simplest investment strategy implemented in the project and serves as the standard comparison for all the other strategies. SIP stands for Systematic Investment Plan, which is a common long-term investing method where a fixed amount of money is invested at regular time intervals regardless of market conditions. In this implementation, the strategy invests a fixed monthly amount on the first available trading day of each month using historical stock price data.

The primary motivation behind the Fixed SIP strategy is consistency and simplicity. Instead of attempting to predict market movements or react to short-term volatility, the strategy continuously purchases shares over time. During periods when prices are low, the fixed investment purchases more shares, while during periods of high prices, it purchases fewer shares.

This behavior naturally averages the purchase cost over long investment periods and reduces the risk associated with trying to time the market.

The Fixed SIP strategy requires only a few parameters: the monthly capital amount, the start year, and the end year for the simulation period. The implementation iterates through the linked list of historical price data, month by month, and purchases fractional shares using the monthly investment amount. The strategy continuously tracks portfolio value, total invested capital, total return, CAGR, maximum drawdown, and total number of trades throughout the simulation period.

One of the major strengths of the Fixed SIP strategy is stability. Because the investment amount never changes, the strategy avoids emotional decision-making and excessive trading. This makes it well-suited for long-term investors seeking predictable and disciplined market exposure. However, the strategy does not attempt to take advantage of major market corrections or strong momentum periods, which may limit its upside potential compared to more adaptive approaches.

Overall, the Fixed SIP strategy provides a reliable benchmark for evaluating the effectiveness of more advanced strategies implemented in the project. By comparing the final portfolio value and risk metrics of other strategies against Fixed SIP, it becomes possible to determine whether more sophisticated algorithms actually provide meaningful performance improvements.

DynamicSIPStrategy

The Dynamic SIP strategy expands upon the traditional SIP model by adjusting investment amounts according to recent market behavior. Instead of investing a constant amount every month, the strategy reacts to market dips and rallies using rolling price analysis. The objective of this strategy is to invest more aggressively when market prices are discounted and reduce investment exposure when prices become excessively high.

The implementation maintains a rolling collection of recent market prices and continuously calculates the percentage decline from recent highs as well as the percentage increase from recent lows. If the current market price falls below a specified dip threshold relative to the recent high, the strategy increases the monthly investment amount using a configurable multiplier. This allows the strategy to purchase additional shares during market corrections. On the other hand, if the market experiences a strong rally above a specified threshold, the strategy reduces the investment amount in order to avoid purchasing shares at inflated prices.

The Dynamic SIP strategy requires several configurable parameters. The dip threshold determines how large a decline must occur before the strategy increases investment. The rally threshold controls when the strategy begins reducing investment during strong upward trends. The multiplier parameter controls how aggressively the investment amount increases during market dips. These parameters allow the strategy to be tuned for different levels of risk tolerance and market responsiveness.

An important design feature of the implementation is the use of budget management constraints. The strategy prevents overspending by limiting monthly investment amounts based

on the remaining simulation budget. Minimum and maximum spending limits are also applied to avoid excessively aggressive or conservative allocation decisions. This ensures that the strategy remains realistic and maintains consistent long-term capital deployment.

The Dynamic SIP strategy attempts to exploit market volatility by purchasing larger quantities of shares during periods of market weakness. In theory, this can lower the average purchase cost and improve long-term returns compared to a traditional fixed investment schedule. However, the strategy also introduces additional risk because incorrect threshold settings or prolonged market declines can negatively impact performance.

To further analyze the behavior of the Dynamic SIP strategy, a parameter sweep feature was implemented as a bonus extension of the project. The parameter sweep automatically tests multiple dip threshold, rally threshold, and multiplier combinations and inserts the resulting portfolio performances into a binary search tree for ranking and comparison. This feature allows the system to automatically determine which threshold configuration produced the highest final portfolio value.

Overall, the Dynamic SIP strategy demonstrates how adaptive investing techniques can potentially outperform fixed investing approaches when combined with historical trend analysis and disciplined budget management.

GoldenCrossStrategy

The Golden Cross strategy is a trend-following trading system based on moving average crossovers. The strategy uses two moving averages: a short-term moving average and a long-term moving average. In most financial applications, the short-term average is typically a 50-day moving average, while the long-term average is a 200-day moving average.

The strategy generates buy and sell signals based on the relationship between these moving averages. A “golden cross” occurs when the short-term moving average rises above the long-term moving average, signaling the beginning of a potential bullish trend. When this crossover occurs, the strategy enters the market and begins purchasing shares. Conversely, when the short-term moving average falls below the long-term moving average, a “death cross” occurs, signaling possible market weakness and causing the strategy to exit or reduce exposure.

The implementation of the Golden Cross strategy relies heavily on the CircularQueue data structure. Rolling windows of recent stock prices are maintained efficiently using fixed-size queues that continuously update the moving average calculations. This allows the strategy to process large historical datasets efficiently without repeatedly recalculating averages from scratch.

The primary advantage of the Golden Cross strategy is its ability to capture large long-term market trends while avoiding prolonged bearish periods. By filtering out short-term market noise, the strategy attempts to remain invested only during strong upward momentum phases. This can potentially reduce drawdowns compared to always remaining fully invested in the market.

However, moving average strategies also have several limitations. Because moving averages are lagging indicators, buy and sell signals may occur well after a trend has already begun. During highly volatile or sideways markets, the strategy may also experience frequent false signals, leading to unnecessary trades and reduced profitability.

Despite these limitations, the Golden Cross strategy remains one of the most widely recognized trend-following systems in financial analysis. Its implementation in the project

demonstrates how rolling financial indicators and queue-based data structures can be combined to create practical algorithmic trading systems.

MomentumStrategy

The Momentum strategy is based on the financial principle that assets demonstrating strong recent performance tend to continue performing well in the near future. Instead of relying on moving averages, the strategy directly measures trailing percentage returns over a specified lookback period and uses this information to make investment decisions.

The strategy calculates the percentage price change between the current market price and the price from a configurable number of trading days earlier. If the calculated momentum exceeds a predefined threshold, the strategy interprets this as evidence of a strong upward trend and enters the market. If the momentum becomes weak or negative, the strategy exits the position and holds cash until market conditions improve.

The implementation uses configurable parameters, including the momentum threshold and the lookback window size. The threshold determines how strong the trend must be before a position is opened, while the lookback period controls how far into historical data the momentum calculation examines. Shorter lookback periods make the strategy more reactive, while longer periods produce smoother but slower signals.

One advantage of the Momentum strategy is its ability to capitalize on sustained market trends. During strong bullish markets, momentum-based systems can produce significant gains by remaining invested throughout the trend duration. Additionally, the strategy may avoid major market crashes by exiting positions when momentum weakens significantly.

However, momentum investing can also be highly sensitive to sudden reversals. Rapid market corrections may generate delayed exits, causing temporary losses before the strategy fully reacts. The strategy may also underperform during sideways or highly volatile markets where clear trends do not exist.

The Momentum strategy demonstrates how historical price analysis and trend detection can be incorporated into algorithmic investment systems. By combining configurable thresholds with systematic trade execution, the strategy provides another method for evaluating long-term market behavior using historical financial datasets.

AI Usage Log - Trading Strategies

Prompts: How do we compare against 12-month rolling high/low; How do I prevent the golden/death cross from being triggered at the initial values; how do we scan backwards ~126 trading days on PriceHistory, a doubly-linked list?

AI Output: Suggested using a fixed-capacity buffer of recent month closes and tracking running high/low for Dynamic SIP; recommended waiting until 50-day and 200-day CircularQueues are full before checking conditions; Suggested a manual prev-pointer to traverse ~126 nodes backward.

Modification: Rewrote DynamicSIP to not use vector<double>, instead using an array and indexing; AI made a separate counter instead of using isFull() for CircularQueue, changed to use isFull(); ReverseIterator should be used instead of a manual prev-pointer, simpler.

Results and Analysis

To simulate four distinct investment strategies and backtest each against real market history, StockSim uses datasets from Yahoo Finance CSV files and simulates a monthly

contribution of \$500 to the tickers SPX, AMZN, and NVDA over the date range 2000-01-01 to 2020-01-01 (20 years). Each investment strategy utilizes the same budget of exactly \$120,000 over the 20-year span. The large data range creates a more level playing field across all strategies, since there are more data points overall. The four investment strategies all output the same five metrics computed after simulation: final value, total return, CAGR (Compound Annual Growth Rate), max drawdown (maximum observed loss), and total number of trades.

SPX 2000–2020	Final Value	Total Return	CAGR	Max Drawdown	Trades
Fixed SIP	\$266,867	122.39%	4.08%	46.25%	240
Dynamic SIP (14.2/5.5/3x)	\$313,330	161.11%	4.92%	41.02%	240
Golden Cross (50/200)	\$258,824	115.69%	3.92%	16.86%	174
6-Month Momentum	\$224,610	87.17%	3.18%	17.85%	120

Table 1: Strategy comparison for simulated **SPX** investment from 2000-2020 (S&P500).

Shown above in Table 1 are our results simulating investment into SPX, representing the S&P 500 dataset, with a monthly contribution of \$500 from 2000-01-01 to 2020-01-01. Based on the final values shown for each strategy, it is clear that for SPX specifically, the best-performing strategy is Dynamic SIP, reaching a final value of \$313,330 or a 161.11% return on investment (ROI). The Dynamic SIP strategy did have a large maximum drawdown at 41.02%, indicating a high level of risk. However, Fixed SIP had an even larger maximum drawdown of 46.25%, implying the most risk of the four strategies.

The reason that the Dynamic SIP strategy excelled more than all other strategies when simulating SPX investment is that it was able to reach its 14.2% dip threshold at many points in SPX's history and invest three times more than Fixed SIP would at these dips. Investing low led to greater gains for the Dynamic SIP strategy when compared to other strategies. Interestingly,

the number of trades that the dynamic and fixed strategies made is equal. This means that the dynamic strategy never overspent, and was able to invest every month of every year.

Golden Cross and 6-month Momentum strategies performed worse than fixed and dynamic strategies in this scenario. Golden Cross did not succeed here because it did not invest in SPX on dramatic dips, and instead sold as the dips were occurring to avoid large losses. This is what the Golden Cross strategy is designed to do: hedge risk. Accordingly, it had the lowest maximum drawdown at 16.86%. The 6-month Momentum strategy struggled for a different reason, namely that it did not make many trades. When SPX was down for long stretches, the Momentum strategy kept its cash rather than investing, thus missing out on potential gains.

AMZN 2000–2020	Final Value	Total Return	CAGR	Max Drawdown	Trades
Fixed SIP	\$3,882,090	3135.08%	18.99%	56.09%	240
Dynamic SIP (14.2/5.5/3x)	\$1,059,410	782.84%	11.50%	55.27%	153
Golden Cross (50/200)	\$990,809	725.68%	11.13%	48.28%	173
6-Month Momentum	\$1,294,960	979.13%	12.63%	32.97%	153

Table 2: Strategy comparison for simulated **AMZN** investment from 2000-2020 (volatile stock).

NVDA 2000–2020	Final Value	Total Return	CAGR	Max Drawdown	Trades
Fixed SIP	\$1,931,060	1509.22%	14.90%	80.90%	240
Dynamic SIP (14.2/5.5/3x)	\$234,720	95.60%	3.41%	79.86%	133
Golden Cross (50/200)	\$2,016,000	1580.00%	15.15%	53.14%	173
6-Month Momentum	\$1,441,170	1100.98%	13.23%	44.07%	163

Table 3: Strategy comparison for simulated **NVDA** investment from 2000-2020 (volatile stock).

Now that we have covered the results for our ETF investment simulation, SPX, we can now analyze how our investment strategies fared when used on volatile “growth stocks”, like

AMZN and NVDA, shown above in Tables 2 and 3. In these two cases, it is clear that the Fixed SIP investment strategy outperforms the Dynamic SIP strategy by an extreme margin. In the case of NVDA, dynamic is actually the worst performer out of the four, at only a 95.6% total return.

To get to the bottom of why the Dynamic SIP strategy performed so poorly, we look at the number of trades completed over the 20-year period. Fixed SIP completes the same number of trades each run, 240, or one trade for each month. The Dynamic SIP strategy invests three times the normal amount when stocks drop more than 14.2% below the prior 12-month rolling high. As NVDA and AMZN are volatile stocks, they fall below their prior 12-month rolling high quite often. As a result, the Dynamic SIP strategy ran out of funds fairly early in the simulation, and only executed 153 trades on AMZN and 133 trades on NVDA. In the case of volatile stocks like NVDA and AMZN, these stocks rose exponentially in spurts, and Dynamic SIP did not trigger larger purchases while the stock prices were continually rising.

The Dynamic SIP strategy parameters can, of course, be tuned to create a far less aggressive investment strategy with a higher rally threshold for executing small trades and a higher dip threshold for executing large trades. With this configuration, the Dynamic SIP strategy would behave more similarly to the Fixed SIP strategy and not run out of money. In our case, however, the parameters of the Dynamic SIP strategy (dip threshold, rally threshold, and multiplier values) were finely tuned to be optimal specifically for SPX. Because the behavior of SPX is so substantially different from the behavior of volatile stocks like NVDA and AMZN, the Dynamic SIP strategy we crafted was poorly configured for these stocks. A Dynamic SIP strategy could certainly work for these volatile stocks, but tuning the three parameter values is necessary for success.

The primary, open-ended research question driving the motivation to create and implement StockSim is to answer whether a Dynamic SIP strategy outperforms a Fixed SIP strategy over 20 years of S&P 500 data. Based on our results in Table 1, the answer to this question is yes; it certainly does. However, it is important to note that the Dynamic SIP strategy used to collect the data shown in Table 1 was, as previously stated, given parameters that would maximize its effectiveness in this 20-year period.

The parameters passed to our Dynamic SIP strategy to make it optimal for investing in SPX were a dip threshold of 14.2%, a rally threshold of 5.5%, and a multiplier of 3. This means that it has fallen at least 14.2% below its prior 12-month rolling high, and a purchase order is placed for 3 times the regular amount (\$1500). On the other hand, when the price of a stock rises more than 5.5%, the Dynamic SIP strategy only purchases half (\$250) of what it normally would. These values were determined by a parameter sweep with 20 different combinations of our three Dynamic SIP parameters, in which the best solution for SPX was chosen.

As we have shown, a Dynamic SIP strategy tuned for the SPX dataset beats a Fixed SIP strategy easily. When this same strategy is applied to volatile stocks, the result is reversed. What we can gather from our results is that a Dynamic SIP strategy is only as good as the parameters passed to it. In order to effectively use a Dynamic SIP strategy, the strategy creator should carefully consider the behavior of the investment in question in order to fine-tune their parameters. Alternatively, one could use a program like StockSim and use past data to find parameters that should theoretically be best for different types of stocks or investments.

AI Usage Log - Results and Analysis

Prompts: SPX seems to be loading only 1540 days of data. The other two load 5031 days.

What is the cause of this issue, and how do we fix it; Case 13 shows a really high CAGR when we have a very large final portfolio value. Please help me identify the root cause and solution; Help me debug DynamicSIPStrategy. It is acting like FixedSIPStrategy for some reason.

AI Output: Traced the SPX data loading error to using long on volume, needed long long; issue with CAGR was a Newton-iteration convergence failure on nthRoot; DynamicSIP's rolling window was simply never being filled, so its dip/rally conditions never went off.

Modification: Project instructions are unclear on whether include `<cmath>` can be used, so I had to find an alternative solution for TradingStrategy.cpp; `std::vector` can only be used for Portfolio according to the instructions. AI accidentally used `std::vector` on the Strategy solutions, which needed to be overwritten.

Conclusion

Through the process of developing, implementing, and using StockSim, our group learned important lessons regarding coordinating multiple headers and source files into a large, robust C++ program. Perhaps the most useful lesson was learning how to diagnose and debug a program of this size and complexity. To fix our code, we had to test in main, then trace each issue back to the relevant function along with its respective header and source files.

Another important lesson we learned was how to utilize AI to assist with developing a large C++ project. Challenges were encountered with AI, mostly due to its lack of the overall context of the program. When AI lacked context, it gave responses that broke the rules of the assignment, as well as implementations of code that did not properly work with the relevant

headers or source files in the project. When utilizing AI, we learned that it is extremely important to ensure the AI understands all of the context and guidelines for the assignment.

Although StockSim successfully executes its designed behavior, it is limited in its overall scope. For example, StockSim does not properly account for real-world non-idealities like taxes, transaction fees, and whether fractional shares are allowed. These were not included in the design requirements, but they could be implemented into the program to make the simulations more robust and realistic. Furthermore, StockSim is also limited in the assets that it tests, receiving data from two stocks (NVDA and AMZN) and one ETF (SPX). Adding more purchasable assets would give us more results to properly analyze and understand how our strategies perform under different conditions.

In future implementations of StockSim, our group would include more assets and more strategies in order to have a more comprehensive program that can be used for a wider variety of potential scenarios. Also, in a future implementation, it would be beneficial to have multiple different Dynamic SIP strategies, each finely tuned for different types of assets. As we observed, the performance of a Dynamic SIP strategy largely depends on its parameters as well as the type of asset it is being used on.

AI Usage Log - Conclusion

No AI used.